

R 패키지의 구조 이해 및 개발

R 프로그래밍 with workflow design



왜 R 패키지 개발을 공부해야 할까?

- **R 패키지**: 함수, 데이터, 문서화를 포함한 코드의 모듈 → 사용 및 공유가 용이
- CRAN, Bioconductor, GitHub 등에서 손쉽게 배포 가능
- **프로젝트 관리 효율화**
  - 여러 스크립트와 중복 코드의 엉킴 방지
  - 기능 모듈화 → 일관성 유지 및 재사용 용이
- **유지보수성과 확장성**
  - 문서화와 버전 관리 체계화
  - 협업과 장기적 코드 관리에 효과적
- **실제 업무과 연구에 필수적**
  - 반복작업 자동화
  - 분석 워크플로우의 표준화 가능

패키지 구조와 메타데이터

R 패키지 구조와 메타데이터란?

- R 패키지: 함수, 데이터, 문서 등을 하나로 묶은 배포 단위
  - 주로 **소스 패키지**(source package)를 중심으로 설명
- 패키지를 만들기 위해서는 **표준 디렉토리 구조**와 **메타데이터 파일**이 필요함
- 패키지 구조의 이해는 유지보수와 배포의 기반이 됨
- 필요한 개발 도구 미리 설치

```
install.packages(c("devtools","usethis","roxygen2","knitr"))
```

R 패키지 기본 디렉토리 구조

구성 요소	설명
DESCRIPTION	메타데이터 저장 (패키지 설명 등)
NAMESPACE	외부에 공개할 함수 정의
R/	R 코드 파일 저장
man/	문서화된 .Rd 파일
data/	데이터셋 저장 (.rda)
inst/	기타 리소스 파일
tests/	테스트 코드 저장 (예: testthat)
vignettes/	사용법 문서 (Rmd 형식)
src/	C/C++ 코드 저장

- 핵심 구성 요소: DESCRIPTION, NAMESPACE, R/, man/, data/, src/

DESCRIPTION 파일

- DESCRIPTION: R이 패키지로 인식하는 **메타데이터 중심 파일**
- 패키지 이름, 설명, 저자, 의존 패키지 등 포함
- devtools 사용 시 자동 생성됨
- 예시:

```
Package: mypackage
Title: A Sample R Package
Version: 0.1.0
Author: Kwan-Young Bak [aut, cre]
Description: This package provides functions for data analysis.
Imports: ggplot2, dplyr
License: GPL-3
```

DESCRIPTION 주요 필드 설명

필드	설명
Package	패키지 이름
Title	짧은 설명
Version	버전 정보 (major.minor.patch)
Author	저자 (역할과 함께 명시 가능)
Maintainer	유지보수 담당자 이메일 포함
Description	상세 설명
Imports	사용하는 외부 패키지
Suggests	선택적 의존 패키지
Encoding	문자 인코딩 (보통 UTF-8)
LazyData	데이터 자동 로딩 여부

- 최근에는 Author, Maintainer 대신 Authors@R 사용 권장

Authors@R 필드 예시

```
Authors@R: c(
  person("Kwan-Young", "Bak",
         email = "kybak@sungshin.ac.kr",
         role = c("aut", "cre"))
)
```

- 역할 태그:
  - "aut": 저자 (Author)
  - "cre": 유지보수자 (Creator)

Imports vs Suggests 필드

- Imports: 필수 의존성. 함수 호출 시 :: 사용 권장
  → 예: ggplot2::ggplot()
- Suggests: 선택적 의존성. 테스트나 예시에만 사용됨
  → 예: testthat, knitr
- usethis 패키지로 간편 추가:

```
usethis::use_package("ggplot2")          # Imports
usethis::use_package("testthat", "Suggests")
```

R 패키지 생성 - 두 가지 방법

1. RStudio에서 프로젝트로 생성
    - [New Project] → [New Directory] → [R Package]
    - 프로젝트와 패키지 구조가 함께 생성됨
2. `devtools::create()` 사용

```
devtools::create("경로/패키지이름")
```

devtools로 패키지 생성하기

```r
install.packages("devtools")
library(devtools)

create("mypkg")   # 현재 작업 디렉토리에 mypkg 폴더 생성
```

- 생성 결과: `.Rproj` 파일 포함
  - `mypkg/`
  - `DESCRIPTION`
  - `NAMESPACE`
  - `R/`
  - `man/`

패키지 설치 및 확인

- RStudio Build 탭 (Environment와 같은 페인에 있음)의 ↗️ Install 버튼 클릭
- 또는 단축키 Ctrl/Cmd + Shift + B
- 패키지 설치 후 테스트:

```
library(mypkg)
```

- 코드 수정 시 → 다시 설치 필요
- 여러 번 반복하며 점검

`roxygen2`를 이용한 문서화와 네임스페이스 관리

왜 문서화가 중요한가?

- 패키지에는 함수와 데이터뿐 아니라 **도움말 문서**도 포함됨
  - `?함수명` 또는 `help()`로 확인 가능
- 잘 문서화된 패키지는:
  - 사용자가 이해하고 활용하기 쉬움
  - **자기 자신도 나중에 유지보수 가능**
  - 협업과 공유에 유리함
- 문서 형식: `man/` 폴더 내 `.Rd` 파일
  - 직접 작성도 가능하지만
  - 대부분 roxygen2 **주석 기반 문서화**를 사용함

roxygen2 문서화 흐름

1. R/ 내부 파일의 함수 위에 roxygen 블록 추가
2. devtools::document() 실행 또는 단축키 Ctrl/Cmd + Shift + D 사용
3. man/*.Rd 파일 자동 생성
4. ?함수명으로 문서 확인 가능

- 즉 R/ 내부에 있는 함수에 주석만 잘 달면 man/에 자동으로 문서가 생김

roxygen 블록 예시

```r
#' Sum of two numbers
#'
#' @param x first number (numeric)
#' @param y second number (numeric)
#' @return sum of two numbers (numeric)
#' @examples
#' add_numbers(3, 5)
#' @export
add_numbers = function(x, y) {
  return(x + y)
}
```

- 함수 위에 `#'`로 시작하는 블록이 필요
- `@export`: 외부에서 사용 가능하도록 공개

자주 사용하는 roxygen 태그

태그	설명
@param	인자의 설명
@return	반환값 설명
@examples	사용 예제
@export	외부 노출 여부
@import	전체 패키지 함수 가져오기
@importFrom	특정 패키지의 함수만 가져오기
@seealso	관련 함수 링크
@details	추가 설명
@author	작성자 정보

문서화 자동화 설정 (RStudio)

- 메뉴: `Build > Configure Build Tools > Generate documentation with Roxygen` 체크
- 옵션: `Automatically roxygenize when running` 활성화
- 효과: **패키지 설치/빌드 시 문서화도 자동 수행**

- [선택 사항] 패키지 전체 문서 만들기
  - `devtools::build_manual()` 실행 시 `.Rd` 파일을 기반으로 PDF 메뉴얼 생성됨 (내부 공유 및 참고용)

네임스페이스와 exports

- R에서 **네임스페이스(namespace)**는 함수, 객체, 데이터 등의 **이름 충돌을 방지**하고 **캡슐화된 독립적인 환경**을 제공하는 시스템
- `exports`는 패키지 외부에서 접근 가능한 함수를 지정함
- 패키지 생성 시 기본적으로 포함되는 NAMESPACE 파일 예시:

```
exportPattern("^[[:alpha:]]+")
```

- 의미: 이름이 영문자로 시작하는 모든 함수를 자동으로 export
- 정규 표현식 기반의 자동 `export` 방식

imports와 NAMESPACE 관리

- imports는 외부 패키지의 함수를 사용하는 방법을 정의함
  - import(pkg): 해당 패키지의 전체 함수 사용
  - importFrom(pkg, fun) 해당 패키지의 특정 함수만 사용
- 패키지 생성 직후에는 의존성이 없기 때문에 NAMESPACE에는 imports 관련 내용이 없음

Rcpp를 활용한 패키지 개발

왜 Rcpp를 패키지 개발에 사용하는가?

- **단순 R 코드보다 더 빠른 계산 성능 확보** : 반복 연산, 대용량 행렬 연산 등에 유리
- **Rcpp 패키지 장점**
  - C++로 작성된 함수들을 R 함수처럼 쉽게 호출 가능
  - 다른 사용자와 코드, 문서, 기능을 함께 공유할 수 있음
  - 패키지 문서화 및 네임스페이스 관리도 자동화 가능
- `sourceCpp()` 로 임시 실행할 수도 있지만 **재사용과 배포에는 패키지화가 필수**

Rcpp 패키지 개발 시작

1. **새 프로젝트 생성 시**: *R package using Rcpp* 선택

- 자동으로 생성되는 구성:
  - `DESCRIPTION`과 `NAMESPACE`에 Rcpp 관련 정보 추가
  - `.gitignore`에 컴파일된 파일 제외 설정
  - `src/` 디렉토리 생성 → C++ 코드 저장용
    - C++ 코드는 `src/` 폴더 아래 `.cpp` 파일로 관리

2. 기존 패키지에 Rcpp 추가하기

- 기존 패키지에 수동 설정하지 말고 아래 함수 사용 권장:

```
usethis::use_rcpp()
```

C++ 코드 작성과 문서화

```
#include <Rcpp.h>
using namespace Rcpp;

// [[Rcpp::export]]
double add(double x, double y) {
  return x + y;
}

// [[Rcpp::export]]
NumericVector square(NumericVector x) {
  return x * x;
}
```

- `// [[Rcpp::export]]`: R에서 직접 호출 가능하게 함
- `.cpp` 파일 안에 바로 `roxygen2` 주석을 달면 문서화 가능
- 주석 기호는 `//'`, 태그 형식은 R과 동일
- R 함수처럼 `@param`, `@return`, `@export` 등을 그대로 사용

설치 후 바로 R에서 C++ 함수 사용하기

- `.cpp` 파일을 패키지 안에 작성하고 install 버튼 또는 `Ctrl/Cmd + Shift + B`로 빌드하면 `Rcpp::sourceCpp()` 없이 바로 함수 사용 가능

```r
library(mycpppkg)

add(2, 3)           # 5 반환
square(1:5)         # c(1, 4, 9, 16, 25) 반환
```

- 여기서 mycpppkg는 본인의 패키지 이름으로 대체하면 됨
- 정리: `.cpp` 파일 작성 → 빌드 → R에서 바로 호출

패키지 배포 과정의 이해

GitHub을 이용한 배포

- R 패키지를 배포하는 가장 간편한 방법: **GitHub 활용**
- GitHub에 올려두기만 해도 누구나 쉽게 설치 가능
- 설치 명령어 예시:

```
devtools::install_github("username/repo")
pak::pak("username/repo")
```

- 예: 챕터 10에서 만든 패키지 설치

```
devtools::install_github("kybak90/BsplineReg")
```

README 파일 작성과 관리 자동화

- GitHub에 패키지를 올릴 때는 `README.md` 파일을 함께 작성하는 것이 일반적
- usethis 패키지의 함수로 시작: `usethis::use_readme_rmd()`
  - `README.Rmd` 템플릿 파일 생성
  - R 코드와 그림 등을 포함한 문서 작성 가능 (R Markdown 기반)
- 배포용 `.md` 파일로 변환: `devtools::build_readme()`
- 커밋 시 주의사항
  - `.md` 파일 상단에 안내 주석 자동 추가: `<!-- README.md is generated from README.Rmd. Please edit that file -->`
  - `.git/hooks/pre-commit` 자동 생성: `README.Rmd` 변경 시 `build_readme()` 실행을 유도
- 문서 내용 수정은 항상 `.Rmd`에서 진행 (`.md`는 자동 생성된 결과물이므로 직접 수정 X)

CRAN 배포

- GitHub은 자유로운 배포, CRAN은 **형식과 조건이 엄격**
- 예:
  - 모든 예제 코드가 5초 이내에 실행되어야 함
  - `\dontrun{}` 또는 예제 간소화 필요
- CRAN 배포는 검증과 관리가 까다롭지만 공식성, 신뢰도, 접근성이 높음